

```

# importing major liberacy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

# importing warning remover
import warnings
warnings.filterwarnings('ignore')

# Sklearn major liberies
from sklearn.model_selection import train_test_split
from sklearn.ensemble import StackingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier,
GradientBoostingClassifier
from xgboost import XGBClassifier

```

Imported major liberies from sklearn for training and testing algorithm

```

df = pd.read_csv('diabetes.csv')

df

{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 768,\n  \"fields\": [\n    {\n      \"column\": \"Pregnancies\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 3,\n        \"min\": 0,\n        \"max\": 17,\n        \"num_unique_values\": 17,\n        \"samples\": [\n          6,\n          1,\n          3\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Glucose\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 31,\n        \"min\": 0,\n        \"max\": 199,\n        \"num_unique_values\": 136,\n        \"samples\": [\n          151,\n          101,\n          112\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"BloodPressure\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 19,\n        \"min\": 0,\n        \"max\": 122,\n        \"num_unique_values\": 47,\n
```

```

{"samples": [{"column": "Pregnancies", "dtype": "number", "std": 86, "min": 0, "max": 17, "num_unique_values": 12, "samples": 46}, {"column": "Glucose", "dtype": "number", "std": 46, "min": 0, "max": 199, "num_unique_values": 51, "samples": 85}, {"column": "BloodPressure", "dtype": "number", "std": 15, "min": 0, "max": 122, "num_unique_values": 12, "samples": 48}, {"column": "SkinThickness", "dtype": "number", "std": 115, "min": 0, "max": 846, "num_unique_values": 186, "samples": 52}, {"column": "Insulin", "dtype": "number", "std": 183, "min": 0, "max": 846, "num_unique_values": 41, "samples": 183}, {"column": "BMI", "dtype": "float", "std": 7.8841603203754405, "min": 0.0, "max": 67.1, "num_unique_values": 248, "samples": 19.9}, {"column": "DiabetesPedigreeFunction", "dtype": "number", "std": 0.33132859501277484, "min": 0.078, "max": 2.42, "num_unique_values": 517, "samples": 1.731}, {"column": "Age", "dtype": "number", "std": 11, "min": 21, "max": 81, "num_unique_values": 52, "samples": 60}, {"column": "Outcome", "dtype": "number", "std": 0, "min": 0, "max": 1, "num_unique_values": 2, "samples": 0}], "type": "dataframe", "variable_name": "df"}

```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 768 entries, 0 to 767
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	int64
3	SkinThickness	768 non-null	int64
4	Insulin	768 non-null	int64
5	BMI	768 non-null	float64

6	DiabetesPedigreeFunction	768	non-null	float64
7	Age	768	non-null	int64
8	Outcome	768	non-null	int64

dtypes: float64(2), int64(7)

memory usage: 54.1 KB

df.isnull().sum()

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

df.describe()

```
{
  "summary": {
    "name": "df",
    "rows": 8,
    "fields": [
      {
        "column": "Pregnancies",
        "properties": {
          "dtype": "number",
          "std": 269.85223453356366,
          "min": 0.0,
          "max": 768.0,
          "num_unique_values": 8,
          "samples": [
            3.8450520833333335,
            3.0,
            768.0
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "Glucose",
        "properties": {
          "dtype": "number",
          "std": 243.73802348295857,
          "min": 0.0,
          "max": 768.0,
          "num_unique_values": 8,
          "samples": [
            120.89453125,
            117.0,
            768.0
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "BloodPressure",
        "properties": {
          "dtype": "number",
          "std": 252.85250535810619,
          "min": 0.0,
          "max": 768.0,
          "num_unique_values": 8,
          "samples": [
            69.10546875,
            72.0,
            768.0
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "SkinThickness",
        "properties": {
          "dtype": "number",
          "std": 263.7684730531098,
          "min": 0.0,
          "max": 768.0,
          "num_unique_values": 7,
          "samples": [
            768.0,
            20.536458333333332,
            32.0
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "Insulin",
        "properties": {
          "dtype": "number",
          "std": 350.26059167945886,
          "min": 0.0,
          "max": 846.0,
          "num_unique_values": 7,
          "samples": [
            768.0,
            79.79947916666667,
            79.79947916666667
          ],
          "semantic_type": ""
        }
      ]
    }
  }
}
```

```

127.25\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\": \"BMI\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 262.05117817552093,\n            \"min\": 0.0,\n            \"max\": 768.0,\n            \"num_unique_values\": 8,\n            \"samples\": [\n              31.992578124999998,\n              32.0,\n              768.0\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          {\n            \"column\": \"DiabetesPedigreeFunction\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 271.3005221658502,\n              \"min\": 0.078,\n              \"max\": 768.0,\n              \"num_unique_values\": 8,\n              \"samples\": [\n                0.47187630208333325,\n                0.3725,\n                768.0\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"Age\",\n              \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 260.1941178528413,\n                \"min\": 11.76023154067868,\n                \"max\": 768.0,\n                \"num_unique_values\": 8,\n                \"samples\": [\n                  33.240885416666664,\n                  29.0,\n                  768.0\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n              },\n              {\n                \"column\": \"Outcome\",\n                \"properties\": {\n                  \"dtype\": \"number\",\n                  \"std\": 271.3865920388932,\n                  \"min\": 0.0,\n                  \"max\": 768.0,\n                  \"num_unique_values\": 5,\n                  \"samples\": [\n                    0.3489583333333333,\n                    1.0,\n                    0.4769513772427971\n                  ],\n                  \"semantic_type\": \"\",\n                  \"description\": \"\"\n                }\n              }\n            }\n          }\n        ],\n        \"type\": \"dataframe\"}

```

```

x = df.drop('Outcome', axis=1)
y = df['Outcome']

```

```

le = LabelEncoder()
y_encoded = le.fit_transform(y)

```

```

x_train,x_test,y_train,y_test =
train_test_split(x,y_encoded,test_size=0.2,random_state=42,stratify=y_
encoded)

```

```

print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)

```

```

(614, 8)
(154, 8)
(614,)
(154,)

```

```

base_learners = [
    ('dt', DecisionTreeClassifier(random_state=42)),
    ('svc', SVC(probability=True, kernel='rbf', random_state=42)),

```

```

    ('lr', LogisticRegression(max_iter=1000))
]
meta_learner = LogisticRegression(max_iter=1000)
stacking_clf = StackingClassifier(
    estimators=base_learners,
    final_estimator=meta_learner,
    cv=5
)
stacking_clf.fit(x_train, y_train)
StackingClassifier(cv=5,
                   estimators=[('dt',
                                DecisionTreeClassifier(random_state=42)),
                                ('svc', SVC(probability=True,
                                             random_state=42)),
                                ('lr',
                                 LogisticRegression(max_iter=1000))],
                   final_estimator=LogisticRegression(max_iter=1000))
y_pred = stacking_clf.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
accuracy
0.7337662337662337

```

	A
	c
	c
	u
	r
	a
Algorit	c
hm	y
Stacki	7
ng	3
Classi	.
fier	3
	8
	%

#Description An ensemble learning technique that combines predictions from multiple base models (**Decision Tree, SVM, and Logistic Regression**) and uses a meta-model (**Logistic Regression**) to make the final prediction. It leverages the strengths of different algorithms to improve overall performance. |

```

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

# random forest classifier
rf_model = RandomForestClassifier(
    n_estimators=100,      # number of trees
    max_depth=None,       # let trees grow fully
    random_state=42,
)

rf_model.fit(x_train,y_train)

RandomForestClassifier(random_state=42)

y_pred = rf_model.predict(x_test)

accuracy = accuracy_score(y_test,y_pred)

accuracy

0.7597402597402597

```

	A
	c
	c
	u
	r
	a
Algorit	c
hm	y
Rando	7
m	5
Forest	.
Classi	9
fier	7
	%

Description

An ensemble algorithm that creates multiple decision trees and combines their predictions using majority voting. It reduces overfitting and improves generalization compared to a single decision tree.

```

# ada boost
ada_model = AdaBoostClassifier(n_estimators=100, random_state=42)

```

```

ada_model.fit(x_train,y_train)
AdaBoostClassifier(n_estimators=100, random_state=42)
y_pred = ada_model.predict(x_test)
accuracy = accuracy_score(y_test,y_pred)
accuracy
0.7922077922077922

```

```

A
c
c
u
r
a
c
y
Algorit
hm
AdaBo 7
ost 9
Classi .
fier 2
2
%
```

Description

A boosting algorithm that sequentially trains weak learners and gives more weight to previously misclassified instances. It focuses on difficult observations and improves prediction accuracy over iterations.

```

# GradientBoost
gb_model = GradientBoostingClassifier(n_estimators=100,
learning_rate=0.1, random_state=42)

gb_model.fit(x_train,y_train)

GradientBoostingClassifier(random_state=42)

y_pred = gb_model.predict(x_test)

accuracy = accuracy_score(y_test,y_pred)

accuracy
0.7532467532467533

```

	A
	c
	c
	u
	r
	a
Algorit	c
hm	y

Gradient	7
Boosting	5
Classifier	.
ng	3
Classi	2
fier	%

Description

A boosting technique that builds trees sequentially, where each new tree corrects the errors made by previous trees. It optimizes performance by minimizing the loss function through gradient descent.

```
# XG boost
xgb_model = XGBClassifier(n_estimators=100, learning_rate=0.1,
max_depth=3, use_label_encoder=False, eval_metric='mlogloss',
random_state=42)

xgb_model.fit(x_train,y_train)

XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None,
early_stopping_rounds=None,
               enable_categorical=False, eval_metric='mlogloss',
               feature_types=None, feature_weights=None, gamma=None,
               grow_policy=None, importance_type=None,
               interaction_constraints=None, learning_rate=0.1,
max_bin=None,
               max_cat_threshold=None, max_cat_to_onehot=None,
               max_delta_step=None, max_depth=3, max_leaves=None,
               min_child_weight=None, missing=nan,
monotone_constraints=None,
               multi_strategy=None, n_estimators=100, n_jobs=None,
               num_parallel_tree=None, ...)

y_pred = xgb_model.predict(x_test)

accuracy = accuracy_score(y_test,y_pred)
```


accuracy

0.7532467532467533

Algorithm	Accuracy
XGBoost Classifier	75.32%
Stacking Classifier	73.38%
Gradient Boosting Classifier	75.32%
Random Forest Classifier	75.97%
AdaBoost Classifier	79.22%

Description

An advanced implementation of gradient boosting that includes regularization, parallel processing, and optimized tree building. It is known for high performance and efficiency on structured datasets.

Model Performance Ranking

AdaBoost Classifier – 79.22% (Best Performing Model)

Random Forest Classifier – 75.97%

Gradient Boosting Classifier – 75.32%

XGBoost Classifier – 75.32%

Stacking Classifier – 73.38%

Business Conclusion

AdaBoost achieved the highest accuracy (79.22%) and is the recommended model for deployment. Random Forest provides a good balance between accuracy and interpretability. Gradient Boosting and XGBoost delivered similar performance, indicating that further hyperparameter tuning may be required to unlock XGBoost's full potential. Stacking Classifier

did not outperform the individual ensemble models, suggesting that the selected base learners may not be sufficiently diverse for effective stacking.

